

Information Security Experiment No. 01

Title

Write a Java/C/C++/Python program that contains a string (char pointer) with a value 'HelloWorld'. The program should AND and XOR each character in this string with 127 and display the result.

Objectives

Implement AND, OR and XOR each character with 127

Outcome

Student should be able to see the result of AND, OR and XOR each character with 127.

Theory:

String

The string is the one-dimensional array of characters terminated by a null ($\backslash 0$).

Each and every character in the array consumes one byte of memory, and the last character must always be 0.

The termination character ($\backslash 0$) is used to identify where the string ends.

In C language string declaration can be done in two ways

1. By char array
2. By string literal

1. By char array

```
char ch[17]={ 'o', 'n', 'l', 'i', 'n', 'e', 's', 'm', 'a', 'r', 't', 't', 'r', 'a', 'i', 'n', 'e', 'r', '\0' };
```

As we know, array index starts from 0, so it will be represented as in the figure given below.

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Character	o	n	l	i	n	e	s	m	a	r	t	t	r	a	i	n	e	r	\0

While declaring string, size is not mandatory. So we can write the above code as given below: `char ch[]={ 'o', 'n', 'l', 'i', 'n', 'e', 's', 'm', 'a', 'r', 't', 't', 'r', 'a', 'i', 'n', 'e', 'r', '\0' }`

2. By string literal

We can also define the string by the string literal in C language. For example:

```
char str[]="onlinesmartrtrainer";
```

In such case, ' $\backslash 0$ ' will be appended at the end of the string by the compiler.

AND Operation

There are two inputs and one output in binary AND operation.

The inputs and result to a binary AND operation can only be 0 or 1.

The binary AND operation will always produce a 1 output if both inputs are 1 and will produce a 0 output if both inputs are 0.

For two different inputs, the output will be 0.

Symbol		Truth Table		
 2-input AND Gate	A	B	Q	
	0	0	0	
	0	1	0	
	1	0	0	
	1	1	1	
Boolean Expression $Q = A \cdot B$, $A \text{ OR } B$				

Fig: AND Gate

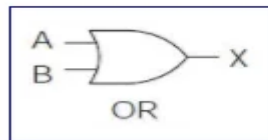
OR Operation

There are two inputs and one output in binary OR operation.

The inputs and result to a binary OR operation can only be 0 or 1.

The OR gate is a mostly used digital logic circuit. The output state of the OR gate will always be low when both of the inputs states is low.

Simply, if any input value in the OR gate is set to 1, then it will always return high-level output(1).



OR gate		
Input A	Input B	Output
0	0	0
1	0	1
0	1	1
1	1	1

Fig: OR Gate

XOR Operation

There are two inputs and one output in binary **XOR** (exclusive **OR**) operation.

It is similar to **ADD** operation which takes two inputs and produces one result i.e. one output.

The inputs and result to a binary **XOR** operation can only be **0** or **1**.

The binary **XOR** operation will always produce a **1** output if either of its inputs is **1** and will produce a **0** output if both of its inputs are **0** or **1**.

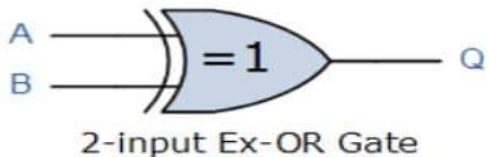
Symbol	Truth Table		
 2-input Ex-OR Gate	A	B	Q
	0	0	0
	0	1	1
	1	0	1
	1	1	0
Boolean Expression $Q = A \text{ XOR } B$			

Fig: XOR Gate

Algorithm:

1. Start
2. Take the input 'hello world' which is assigned to variable 'str'
3. Perform AND operation between the string and 127.
4. Perform OR operation between the string and 127
5. Perform XOR operation between the string and 127
6. Then print the result
7. Stop.

Conclusion:

Thus we have studied AND, OR and XOR gate operation on string to identify the encryption process.

Experiment No. 02

Title

Write a Java/C/C++/Python program to perform encryption and decryption using the method of Transposition technique.

Objective

Learn how to Perform Encryption and Decryption using method of transposition technique.

Problem Definition:

Perform Encryption and Decryption using method of transposition technique.

Outcome

After completion of this assignment students will be able to understand the Perform Encryption and Decryption using method of transposition technique.

Software Requirements:

Python 3

Hardware Requirements:

PC, 2GB RAM, 500 GB HDD

Theory

Transposition Techniques

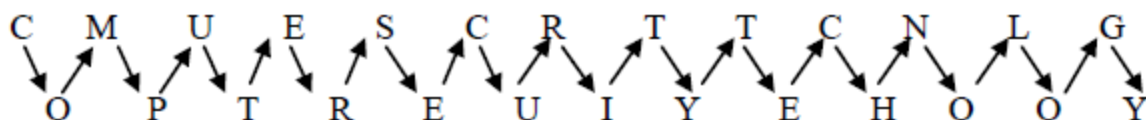
Rail Fence Technique

In Rail fence cipher, techniques are essentially Transposition Ciphers and generated by rearrangement of characters in the plaintext. The characters of the plain text string are arranged in the form of a rail-fence as follows.

Given **Plain text** is - COMPUTER SECURITY TECHNOLOGY

Rail Fence Technique algorithm:

1. Write down the plain text message as a sequence of diagonals.
2. Read the plain text written in Step-1 as a sequence of rows. Example: plain text = COMPUTER SECURITY TECHNOLOGY is converted to cipher text with this help of Rail Fence Technique with dual slope.



Cipher Text is - CMUESCRTTCNLGOPTREUIYEHOOY

Columnar Transposition

Following are two types of Columnar Transposition

Simple Columnar Transposition

The columnar transposition cipher is a transposition cipher that follows a simple rule for mixing up the characters in the plaintext to form the cipher-text. It can be combined with other ciphers, such as a substitution cipher, the combination of which can be more difficult to break than either cipher on its own.

The cipher uses a columnar transposition to greatly improve its security.

Algorithm:

1. The message is written out in rows of a fixed length.
2. Read out again column by column according to given order or in random order.
3. According to order write cipher text.

Example

The key for the columnar transposition cipher is a keyword e.g. ORANGE. The row length that is used is the same as the length of the keyword.

To encrypt a below **Plain Text** - COMPUTER PROGRAMMING

O	R	A	N	G	E
C	O	M	P	U	T
E	R	P	R	O	G
R	A	M	M	I	N
G	L	E	X	X	M

In the above example, the plaintext has been padded so that it neatly fits in a rectangle. This is known as a regular columnar transposition. An irregular columnar transposition leaves these characters blank, though this makes decryption slightly more difficult. The columns are now reordered such that the letters in the key word are ordered alphabetically.

5	6	1	4	3	2
O	R	A	N	G	E
C	O	M	P	U	T
E	R	P	R	O	G
R	A	M	M	I	N
G	L	E	X	X	M

The Encrypted text or **Cipher Text** is: MPMET GNMUO IXPRM XCERG ORAL
(Written in blocks of five)

Double Columnar Transposition

A single Columnar Transposition can be attacked by guessing possible column lengths by writing the message from columns (with the wrong order because the key is unknown) and then trying to get the possible message.

Therefore to make it stronger, a double transposition was used. This is simple columnar transposition technique applied twice. Here the same key can be used for both transpositions or two different keys can be used.

For Example – Plain Text: - WELCOME HOME Key:-PLAYER (with length 6)

Round 1

1	2	3	4	5	6
W	E	L	C	O	M
E	H	O	M	E	

Now read it with some random order of (4,6,1,2,5,3) = “CMMWEEHOELO”

Round 2 now “CMMWEEHOELO” this will be next cipher Text

1	2	3	4	5	6
C	M	M	W	E	E
H	O	E	L	O	

Again read with the order of (4,6,1,2,5,3) = “WLECHMOEOME”

Algorithm:-

1. Write the plain text message row-by-row in a rectangle of a predefined size
2. Read the message column-by-column in any random order.
3. The message thus obtained is cipher text message of round one
4. Repeat step 1 to 3 as many times as desired.

Conclusion

Thus we learn that how to Perform Encryption and Decryption using method of transposition technique.

IS Experiment No. 03

Aim:

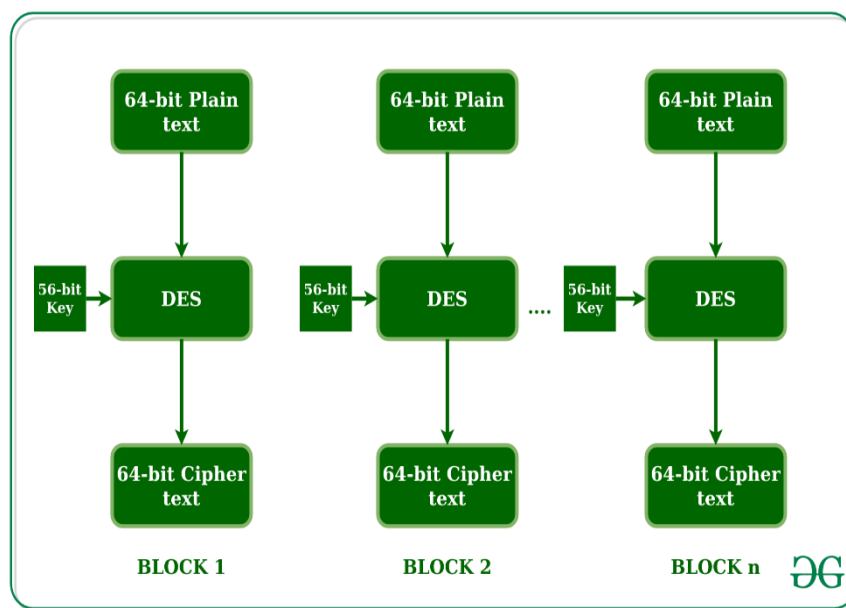
Write a Java/C/C++/Python program to implement DES algorithm.

Theory:

The Data Encryption Standard (DES) is the classic among the symmetric block cipher algorithm. DES was developed in the 1970's as a US-government standard for protecting non-classified information and was published as Federal Information Processing Standard.

Data encryption standard (DES) has been found vulnerable against very powerful attacks and therefore, the popularity of DES has been found slightly on the decline.

DES is a block cipher and encrypts data in blocks of size of 64 bits each, which means 64 bits of plain text goes as the input to DES, which produces 64 bits of ciphertext. The same algorithm and key are used for encryption and decryption, with minor differences. The key length is 56 bits. The basic idea is shown in the figure.



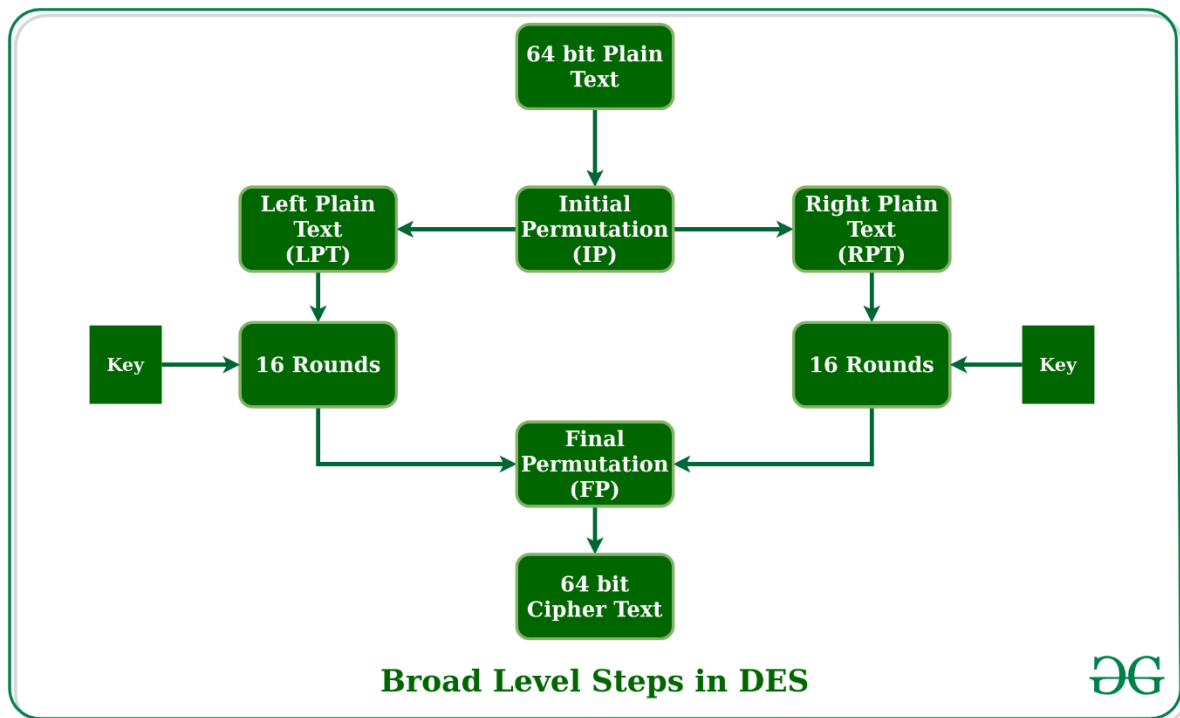
We have mentioned that DES uses a 56-bit key. Actually, the initial key consists of 64 bits. However, before the DES process even starts, every 8th bit of the key is discarded to produce a 56-bit key. That is bit positions 8, 16, 24, 32, 40, 48, 56, and 64 are discarded.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48
49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64

Figure - discarding of every 8th bit of original key

Thus, the discarding of every 8th bit of the key produces a 56-bit key from the original 64-bit key.

DES is based on the two fundamental attributes of cryptography: substitution (also called confusion) and transposition (also called diffusion). DES consists of 16 steps, each of which is called a round. Each round performs the steps of substitution and transposition.



Steps:-

1. 64-Bit plain text block is handed over to an Initial Permutation (IP) function
2. Initial Permutation (IP) is performed on plain text
3. IP produces two halves of permuted block
Left plain text (LPT) and Right plain text (RPT)
4. Each LPT and RPT goes through 16 rounds of encryption process, each with its own key.
5. In the end LPT and RPT are rejoined and final Permutation (FP) is performed on combined block
6. The result is 64 bit cipher text.

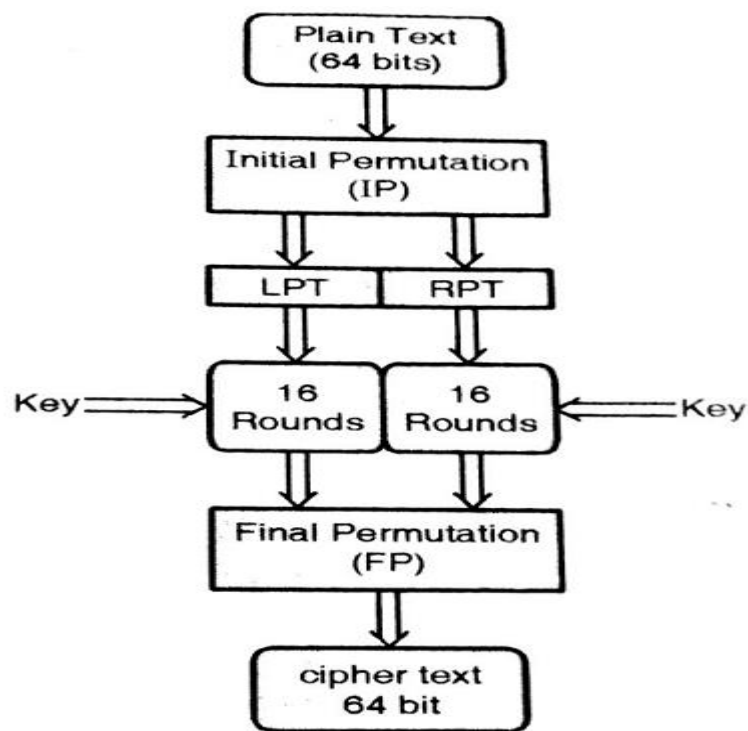


Fig: - Steps in DES

Initial Permutation (IP)

As we have noted, the initial permutation (IP) happens only once and it appens before the first round. It suggests how the transposition in IP should proceed, as shown in figure.

For example, it says that the IP replaces the first bit of the original plain text block with the 58th bit of the original plain text, the second bit with the 50th bit of the original plain text block, and so on.

This is nothing but jugglery of bit positions of the original plain text block. the same rule applies to all the other bit positions shown in the figure.

58	50	42	34	26	18	10	2	60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6	64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1	59	51	43	35	27	19	11	3
61	33	45	37	29	21	13	5	63	55	47	39	31	23	15	7

Figure - Initial permutation table

As we have noted after IP is done, the resulting 64-bit permuted text block is divided into two half blocks. Each half-block consists of 32 bits, and each of the 16 rounds, in turn, consists of the broad level steps outlined in the figure.

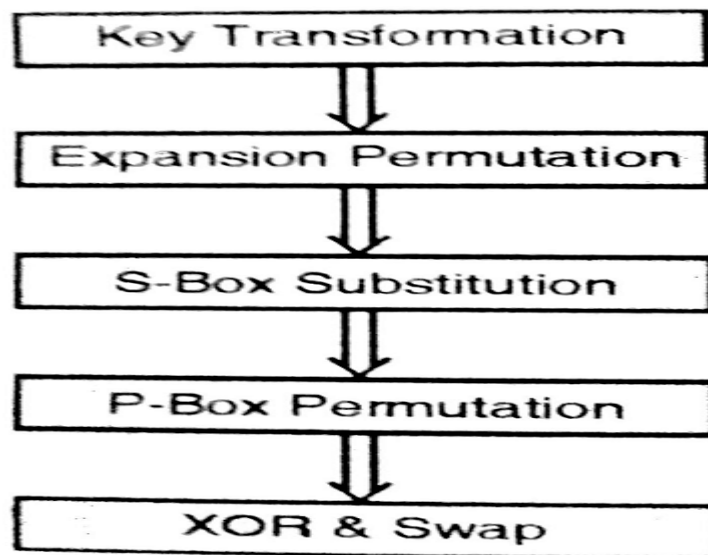


Fig: -Rounds in DES

Step-1: Key transformation –

We have noted initial 64-bit key is transformed into a 56-bit key by discarding every 8th bit of the initial key. Thus, for each a 56-bit key is available. From this 56-bit key, a different 48-bit Sub Key is generated during each round using a process called key transformation.

For this, the 56-bit key is divided into two halves, each of 28 bits. These halves are circularly shifted left by one or two positions, depending on the round.

For example, if the round numbers 1, 2, 9, or 16 the shift is done by only position for other rounds, the circular shift is done by two positions. The number of key bits shifted per round is shown in the figure.

Round	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
#key bits shifted	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

Figure - number of key bits shifted per round

After an appropriate shift, 48 of the 56 bits are selected. for selecting 48 of the 56 bits the table is shown in the figure given below. For instance, after the shift, bit number 14 moves on the first position, bit number 17 moves on the second position, and so on. If we observe the table carefully, we will realize that it contains only 48-bit positions. Bit number 18 is discarded (we will not find it in the table), like 7 others, to reduce a 56-bit key to a 48-bit key. Since the key transformation process involves permutation as well as a selection of a 48-bit subset of the original 56-bit key it is called Compression Permutation.

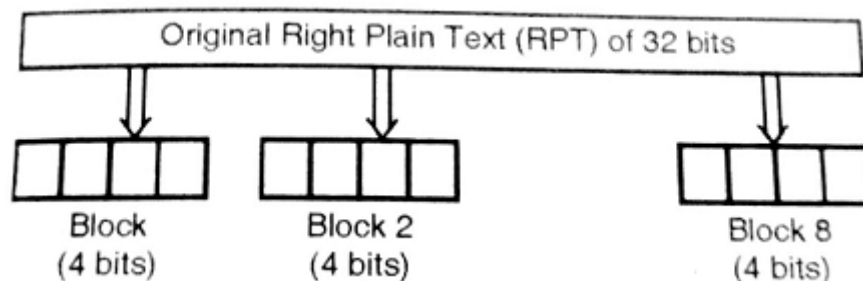
14	17	11	24	1	5	3	28	15	6	21	10
23	19	12	4	26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40	51	45	33	48
44	49	39	56	34	53	46	42	50	36	29	32

Figure - compression permutation

Because of this compression permutation technique, a different subset of key bits is used in each round. That makes DES not easy to crack.

Expansion Permutation

During Expansion permutation the RPT is expanded from 32 bits to 48 bits. The 32-bit RPT is divided into 8 blocks, with each block consisting of 4- bits.



Each 4-bits block of the previous step is then expanded to a corresponding 6-bit block. Per 4-bit block, 2 more bits are added. They are the repeated 1st and 4th bits of the 4-bit block.

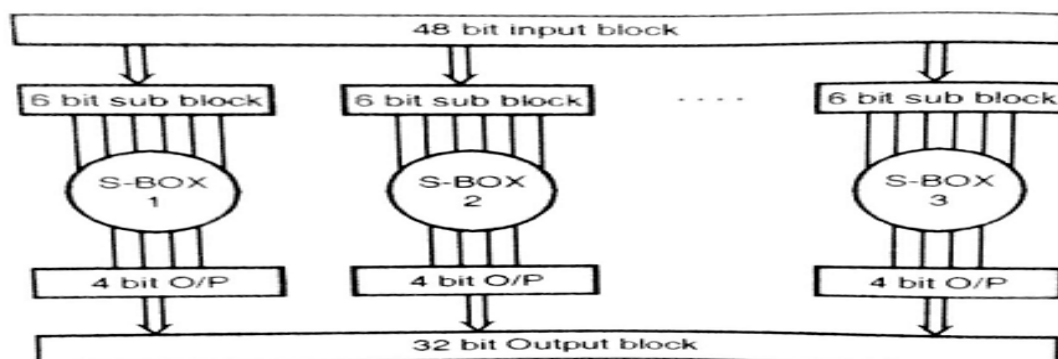
The 2nd and 3rd bits are written as they were in the input.

The 48 bit key is XORed with the 48-bit RPT and the resulting output is given to the next step.

S-box substitution

It accepts the 48-bits input from the XOR operation involving the compressed key and expanded RPT and produces 32-bit output using the substitution techniques.

Each of the 8 S-boxes has a 6-bit input and a 4-bit output as shown below.



P-box permutation

The output of S-box consists of 32 bits. These 32 bits are permuted using a P-box. It involves simple permutation.

For eg., a 16 in the first block indicates that the bit at position 16 of the original input moves to bit at position 1 in the output and a 10 in the block number 16 indicates that the bit at the position 10 of the original input moves to bit at position 16 in the output.

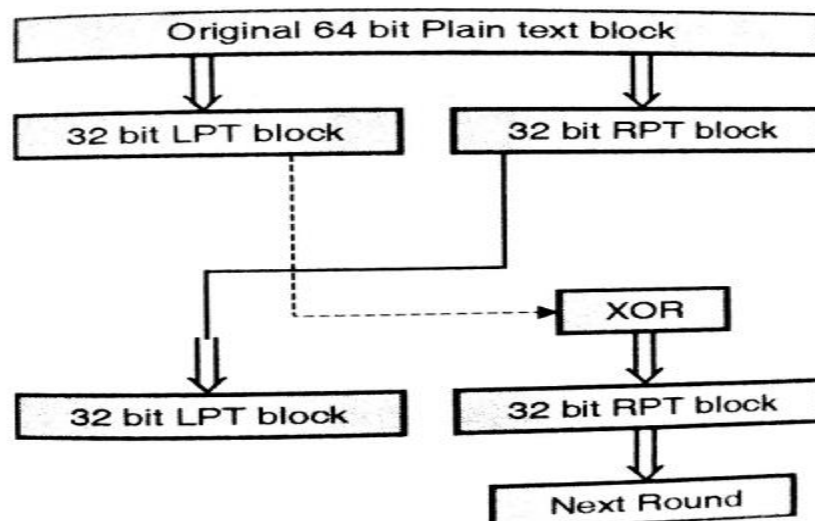
16	7	20	21	29	12	28	17	1	15	23	26	5	18	31	10
2	8	24	14	32	27	3	9	19	13	30	6	22	11	4	25

XOR and swap

The LPT of the initial 64-bit plain text block is XORED with the output produced by P-box permutation.

The result of this XOR operation becomes the new RPT.

The old right half (RPT) becomes the new left half, in the process of swapping.



Final Permutation

At the end of 16 rounds, the Final Permutation is performed only once (simple transposition).

The output of Final Permutation is the 64 bit encryption block.

Conclusion:

Thus we have studied encryption and decryption using DES algorithm

Experiment no. 05

Aim:

Write a Java/C/C++/Python program to implement RSA algorithm

Theory:

RSA (Rivest–Shamir–Adleman) algorithm is asymmetric cryptography algorithm. The acronym "RSA" comes from the surnames of Ron Rivest, Adi Shamir and Leonard Adleman, who publicly described the algorithm in 1977. Asymmetric actually means that it works on two different keys i.e. Public Key and Private **Key**. As the name describes that the Public Key is given to everyone and Private key is kept private.

An example of asymmetric cryptography :

1. A client (for example browser) sends its public key to the server and requests for some data.
2. The server encrypts the data using client's public key and sends the encrypted data.
3. Client receives this data and decrypts it.

Since this is asymmetric, nobody else except browser can decrypt the data even if a third party has public key of browser.

The idea! The idea of RSA is based on the fact that it is difficult to factorize a large integer. The public key consists of two numbers where one number is multiplication of two large prime numbers. And private key is also derived from the same two prime numbers. So, if somebody can factorize the large number, the private key is compromised. Therefore, encryption strength totally lies on the key size and if we double or triple the key size, the strength of encryption increases exponentially. RSA keys can be typically 1024 or 2048

bits long, but experts believe that 1024-bit keys could be broken in the near future. But till now it seems to be an infeasible task.

Let us learn the mechanism behind RSA algorithm :

Generating Public Key :

1. Select two prime no's. Suppose **P = 53 and Q = 59.**
2. Now First part of the Public key : **n = P*Q = 3127.**
3. We also need a small exponent say **e** :
4. But e Must be
5.
 - a. An integer.
 - b. Not be a factor of n.
 - c. $1 < e < \Phi(n)$ [$\Phi(n)$ is discussed below],
 - d. Let us now consider it to be equal to 3.
6. Our Public Key is made of n and e
 - a. >> **Generating Private Key :**
7. We need to calculate $\Phi(n)$:
8. Such that **$\Phi(n) = (P-1)(Q-1)$**
9. so, $\Phi(n) = 3016$
10. Now calculate Private Key, **d** :
11. **$d = (k * \Phi(n) + 1) / e$** for some integer k
12. For k = 2, value of d is 2011.
 - a. Now we are ready with our – Public Key (n = 3127 and e = 3)
and Private Key(d = 2011)
 - b. Now we will encrypt “**HI**” :
13. Convert letters to numbers : H = 8 and I = 9

14. Thus **Encrypted Data** $c = 89^e \bmod n$.

15. Thus our Encrypted Data comes out to be 1394

16. Now we will decrypt **1394** :

17. **Decrypted Data** $= c^d \bmod n$.

18. Thus our Encrypted Data comes out to be 89

19. **8 = H and I = 9 i.e. "HI"**.

Conclusion:

Thus we have studied RSA (Rivest–Shamir–Adleman) algorithm, a asymmetric cryptography algorithm.

Experiment no. 05

Aim: Write a program to implement the Diffie-Hellman Key Exchange mechanism using HTML and JavaScript. Consider the end user as one of the parties (Alice) and the JavaScript application as other party (bob).

Theory:

Diffie-Hellman algorithm:

The Diffie-Hellman algorithm is being used to establish a shared secret that can be used for secret communications while exchanging data over a public network using the elliptic curve to generate points and get the secret key using the parameters.

- For the sake of simplicity and practical implementation of the algorithm, we will consider only 4 variables, one prime P and G (a primitive root of P) and two private values a and b .
- P and G are both publicly available numbers. Users (say Alice and Bob) pick private values a and b and they generate a key and exchange it publicly. The opposite person receives the key and that generates a secret key, after which they have the same secret key to encrypt.

Step-by-Step explanation is as follows:

Alice	Bob
Public Keys available = P, G	Public Keys available = P, G
Private Key Selected = a	Private Key Selected = b
Key generated =	Key generated =
Exchange of generated keys takes place	
Key received = y	key received = x
Generated Secret Key =	Generated Secret Key =
Algebraically, it can be shown that	

Alice	Bob
Users now have a symmetric secret key to encrypt	

Example:

Step 1: Alice and Bob get public numbers $P = 23$, $G = 9$

Step 2: Alice selected a private key $a = 4$ and

Bob selected a private key $b = 3$

Step 3: Alice and Bob compute public values

Alice: $x = (9^4 \bmod 23) = (6561 \bmod 23) = 6$

Bob: $y = (9^3 \bmod 23) = (729 \bmod 23) = 16$

Step 4: Alice and Bob exchange public numbers

Step 5: Alice receives public key $y = 16$ and

Bob receives public key $x = 6$

Step 6: Alice and Bob compute symmetric keys

Alice: $k_a = y^a \bmod p = 6^{16} \bmod 23 = 9$

Bob: $k_b = x^b \bmod p = 6^3 \bmod 23 = 9$

Step 7: 9 is the shared secret.

Conclusion:

Thus we have studied Diffie-Hellman Key Exchange mechanism using HTML and JavaScript